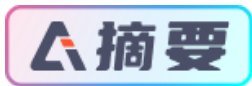


FreeRTOS中断管理 基于STM32



本文详细阐述了异常与中断的基本概念，探讨了中断管理在FreeRTOS中的运作机制，包括中断的触发、优先级、嵌套、延迟响应以及中断服务实验示例。通过实例展示了如何在Cortex-M系列处理器上配置中断、编写中断服务程序，并分析了中断管理在实际项目中的应用场景。

文章目录

[一、异常与中断的基本概念](#)

[二、中断的介绍](#)

[三、和中断相关的名词解释](#)

[四、中断管理的运作机制](#)

[五、中断延迟的概念](#)

[六、中断管理的应用场景](#)

[七、中断管理讲解](#)

[八、中断管理实验](#)

[九、中断管理实验现象](#)

一、异常与中断的基本概念

异常是导致处理器脱离正常运行转向执行特殊代码的任何事件，如果不及时进行处理，轻则系统出错，重则会导致系统毁灭性瘫痪。所以正确地处理异常，避免错误的发生是提高软件鲁棒性（稳定性）非常重要的一环，对于实时系统更是如此。

异常是指任何打断处理器正常执行，并且迫使处理器进入一个由有特权的特殊指令执行的事件。异常通常可以分成两类：同步异常和异步异常。由内部事件（像处理器指令运行产生的事件）引起的异常称为同步异常，例如造成被零除的算术运算引发一个异常，又如在某些处理器体系结构中，对于确定的数据尺寸必须从内存的偶数地址进行读和写操作。从一个奇数内存地址的读或写操作将引起存储器存取一个错误事件并引起一个异常（称为校准异常）。

异步异常主要是指由于外部异常源产生的异常，是一个由外部硬件装置产生的事件引起的异步异常。同步异常不同于异步异常的地方是事件的来源，同步异常事件是由于执行某些指令而从处理器内部产生的，而异步异常事件的来源是外部硬件装置。例如按下设备某个按钮产生的事件。同步异常与异步异常的区别还在于，同步异常触发后，系统必须立刻进行处理而不能够依然执行原有的程序指令步骤；而异步异常则可以延缓处理甚至是忽略，例如按键中断异常，虽然中断异常触发了，但是系统可以忽略它继续运行（同样也忽略了相应的按键事件）。

中断，中断属于异步异常。所谓中断是指中央处理器CPU正在处理某件事的时候，外部发生了某一事件，请求CPU迅速处理，CPU暂时中断当前的工作，转入处理所发生的事件，处理完后，再回到原来被中断的地方，继续原来的工作，这样的过程称为中断。

中断能打断任务的运行，无论该任务具有什么样的优先级，因此中断一般用于处理比较紧急的事件，而且只做简单处理，例如标记该事件，在使用FreeRTOS系统时，一般建议使用信号量、消息或事件标志组等标志中断的发生，将这些内核对象发布给处理任务，处理任务再做具体处理。

通过中断机制，在外设不需要CPU介入时，CPU可以执行其他任务，而当外设需要CPU时通过产生中断信号使CPU立即停止当前任务转而来响应中断请求。这样可以使CPU避免把大量时间耗费在等待、查询外设状态的操作上，因此将大大提高系统实时性及执行效率。

此处读者要知道一点，FreeRTOS 源码中有许多处临界段的地方，临界段虽然保护了关键代码的执行不被打断，但也会影响系统的实时，任何使用了操作系统的中断响应都不会比裸机快。比如，某个时候有一个任务在运行中，并且该任务部分程序将中断屏蔽掉，也就是进入临界段中，这个时候如果有一个紧急的中断事件被触发，这个中断就会被挂起，不能得到及时响应，必须等到中断开启才可以得到响应，如果屏蔽中断时间超过了紧急中断能够容忍的限度，危害是可想而知的。所以，操作系统的中断在某些时候会有适当的中断延迟，因此调用中断屏蔽函数进入临界段的时候，也需快进快出。当然 FreeRTOS 也能允许一些高优先级的中断不被屏蔽掉，能够及时做出响应，不过这些中断就不受系统管理，也不允许调用 FreeRTOS 中与中断相关的任何 API 函数接口。

FreeRTOS 的中断管理支持：

- 开/关中断。
- 恢复中断。
- 中断使能。
- 中断屏蔽。
- 可选择系统管理的中断优先级。

二、中断的介绍

与中断相关的硬件可以划分为三类：外设、中断控制器、CPU 本身。

外设：当外设需要请求 CPU 时，产生一个中断信号，该信号连接至中断控制器。

中断控制器：中断控制器是 CPU 众多外设中的一个，它一方面接收其他外设中断信号的输入，另一方面，它会发出中断信号给 CPU。可以通过对中断控制器编程实现对中断源的优先级、触发方式、打开和关闭源等设置操作。在 Cortex-M 系列控制器中常用的中断控制器是 NVIC（内嵌向量中断控制器 Nested Vectored Interrupt Controller）。

CPU：CPU 会响应中断源的请求，中断当前正在执行的任务，转而执行中断处理程序。NVIC 最多支持 240 个中断，每个中断最多 256 个优先级。

三、和中断相关的名词解释

中断号：每个中断请求信号都会有特定的标志，使得计算机能够判断是哪个设备提出的中断请求，这个标志就是中断号。

中断请求：“紧急事件”需向 CPU 提出申请，要求 CPU 暂停当前执行的任务，转而处理该“紧急事件”，这一申请过程称为中断请求。

中断优先级：为使系统能够及时响应并处理所有中断，系统根据中断时间的重要性和紧迫程度，将中断源分为若干个级别，称作中断优先级。

中断处理程序：当外设产生中断请求后，CPU 暂停当前的任务，转而响应中断申请，即执行中断处理程序。

中断触发：中断源发出并送给 CPU 控制信号，将中断触发器置“1”，表明该中断源产生了中断，要求 CPU 去响应该中断，CPU 暂停当前任务，执行相应的中断处理程序。中断触发类型：外部中断申请通过一个物理信号发送到 NVIC，可以是电平触发或边沿触发。

中断向量：中断服务程序的入口地址。

中断向量表：存储中断向量的存储区，中断向量与中断号对应，中断向量在中断向量表中按照中断号顺序存储。

临界段：代码的临界段也称为临界区，一旦这部分代码开始执行，则不允许任何中断打断。为确保临界段代码的执行不被中断，在进入临界段之前须关中断，而临界段代码执行完毕后，要立即开中断。

四、中断管理的运作机制

当中断产生时，处理机将按如下的顺序执行：

- 1. 保存当前处理机状态信息
- 2. 载入异常或中断处理函数到 PC 寄存器
- 3. 把控制权转交给处理函数并开始执行
- 4. 当处理函数执行完成时，恢复处理器状态信息
- 5. 从异常或中断中返回到前一个程序执行点

中断使得 CPU 可以在事件发生时才给予处理，而不必让 CPU 连续不断地查询是否有相应的事件发生。通过两条特殊指令：关中断和开中断可以让处理器不响应或响应中断，在关闭中断期间，通常处理器会把新产生的中断挂起，当中断打开时立刻进行响应，所以会有适当的延时响应中断，故用户在进入临界区的时候应快进快出。

中断发生的环境有两种情况：在任务的上下文中，在中断服务函数处理上下文中。

● 任务在工作的时候，如果此时发生了一个中断，无论中断的优先级是多大，都会打断当前任务的执行，从而转到对应的中断服务函数中执行，其过程具体见图1

图 1、(1) (3)：在任务运行的时候发生了中断，那么中断会打断任务的运行，那么操作系统将先保存当前任务的上下文环境，转而去处理中断服务函数。

图1、(2) (4)：当且仅当中断服务函数处理完的时候才恢复任务的上下文环境，继续运行任务。

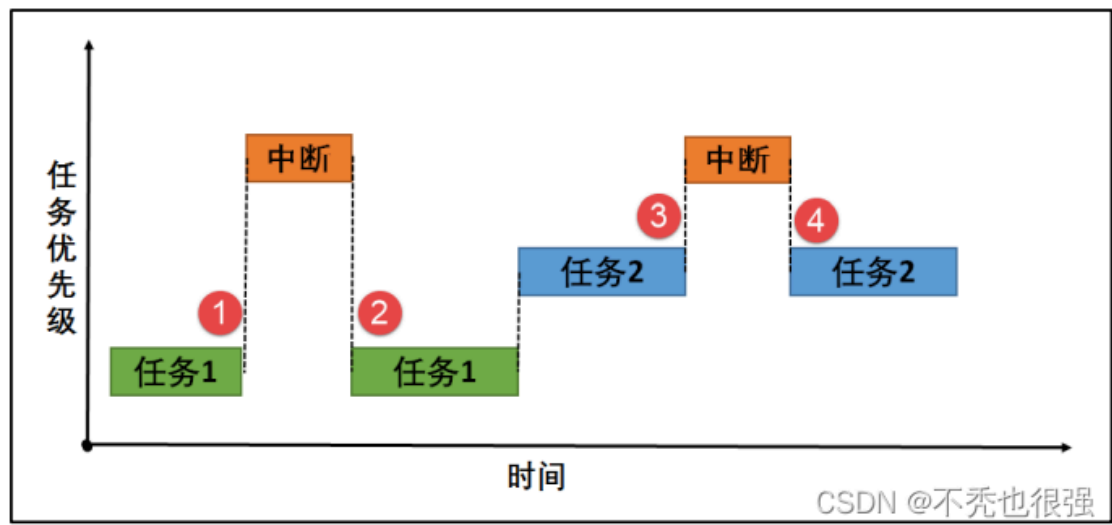


图1中断发生在任务上下文

● 在执行中断服务例程的过程中，如果有更高优先级别的中断源触发中断，由于当前处于中断处理上下文环境中，根据不同的处理器构架可能有不同的处理方式，比如新的中断等待挂起直到当前中断处理离开后再行响应；或新的高优先级中断打断当前中断处理过程，而去直接响应这个更高优先级的新中断源。后面这种情况，称之为中断嵌套。在硬实时环境中，前一种情况是不允许发生的，不能使响应中断的时间尽量短。而在软件处理（软实时环境）上，FreeRTOS 允许中断嵌套，即在一个中断服务例程期间，处理器可以响应另外一个优先级更高的中断，过程如图 2 所示。

当中断 1 的服务函数在处理的时候发生了中断 2，由于中断 2 的优先级比中断 1 更高，所以发生了中断嵌套，那么操作系统将先保存当前中断服务函数的上下文环境，并且转向处理中断 2，当且仅当中断 2 执行完的时候图 (2)，才能继续执行中断 1。

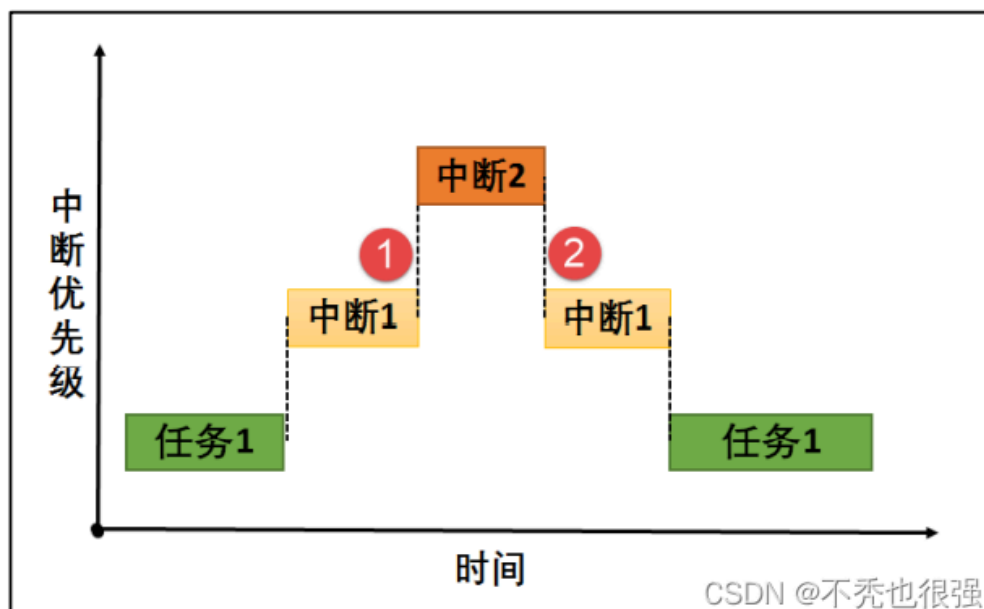


图2中断嵌套发生

五、中断延迟的概念

即使操作系统的响应很快了，但对于中断的处理仍然存在着中断延迟响应的问题，我们称之为中断延迟(Interrupt Latency)。

中断延迟是指从硬件中断发生到开始执行中断处理程序第一条指令之间的这段时间。也就是：系统接收到中断信号到操作系统作出响应，并完成换到转入中断服务程序的时间。也可以简单地理解为：（外部）硬件（设备）发生中断，到系统执行中断服务子程序（ISR）的第一条指令的时间。

中断的处理过程是：外界硬件发生了中断后，CPU 到中断处理器读取中断向量，并且查找中断向量表，找到对应的中断服务子程序（ISR）的首地址，然后跳转到对应的 ISR 去做相应处理。这部分时间，我称之为：识别中断时间。

在允许中断嵌套的实时操作系统中，中断也是基于优先级的，允许高优先级中断抢断正在处理的低优先级中断，所以，如果当前正在处理更高优先级的中断，即使此时有低优先级的中断，也系统不会立刻响应，而是等到高优先级的中断处理完之后，才会响应。而即使在不支持中断嵌套，即中断是没有优先级的，中断是不允许被中断的，所以，如果当前系统正在处理一个中断，而此时另一个中断到来了，系统也是不会立即响应的，而只是等处理完当前的中断之后，才会处理后来的中断。此部分时间，我称其为：等待中断打开时间。

在操作系统中，很多时候我们会主动进入临界段，系统不允许当前状态被中断打断，故而在临界区发生的中断会被挂起，直到退出临界段时候打开中断。此部分时间，我称其为：关闭中断时间。

中断延迟可以定义为，从中断开始的时刻到中断服务例程开始执行的时刻之间的时间段。中断延迟 = 识别中断时间 + [等待中断打开时间] + [关闭中断时间]。

六、中断管理的应用场景

中断在嵌入式处理器中应用非常之多，没有中断的系统不是一个好系统，因为有中断，才能启动或者停止某件事情，从而转去做另一件事情。我们可以举一个日常生活中的例子来说明，假如你正在给朋友写信，电话铃响了，这时你放下手中的笔去接电话，通话完毕再继续写信。这个例子就表现了中断及其处理的过程：电话铃声使你暂时中止当前的工作，而去处理更为急需处理的事情——接电话，当把急需处理的事情处理完毕之后，再回过头来继续原来的事情。在这个例子中，电话铃声就可以称为“中断请求”，而你暂停写信去接电话就叫作“中断响应”，那么接电话的过程就是“中断处理”。由此我们可以看出，在计算机执行程序的过程中，由于出现某个特殊情况(或称为“特殊事件”)，使得系统暂时中止现行程序，而转去执行处理这一特殊事件的程序，处理完毕之后再回到原来程序的中断点继续向下执行。

为什么说没有中断的系统不是好系统呢？我们可以再举一个例子来说明中断的作用。假设有一个朋友来拜访你，但是由于不知何时到达，你只能在门口等待，于是什么事情也干不了；但如果在门口装一个门铃，你就不必在门口等待而可以在

家里去做其他的工作，朋友来了按门铃通知你，这时你才中断手中的工作去开门，这就避免了不必要的等待。CPU 也是一样，如果时间都浪费在查询的事情上，那这个 CPU 啥也干不了，要他何用。在嵌入式系统中合理利用中断，能更好利用 CPU 的资源。

七、中断管理讲解

ARM Cortex-M 系列内核的中断是由硬件管理的，而 FreeRTOS 是软件，它并不接管由硬件管理的相关中断（接管简单来说就是，所有的中断都由 RTOS 的软件管理，硬件来了中断时，由软件决定是否响应，可以挂起中断，延迟响应或者不响应），只支持简单的开关中断等，所以 FreeRTOS 中的中断使用其实跟裸机差不多的，需要我们自己配置中断，并且使能中断，编写中断服务函数，在中断服务函数中使用内核 IPC 通信机制，一般建议使用[信号量](#)、消息或事件标志组等标志事件的发生，将事件发布给处理任务，等退出中断后再由相关处理任务具体处理中断。

用户可以自定义配置系统可管理的最高中断优先级的宏定义 configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY，它是用于配置内核中的 basepri 寄存器的，当 basepri 设置为某个值的时候，NVIC 不会响应比该优先级低的中断，而优先级比之更高的中断则不受影响。就是说当这个宏定义配置为 5 的时候，中断优先级数值在 0、1、2、3、4 的这些中断是不受 FreeRTOS 屏蔽的，也就是说即使在系统进入临界段的时候，这些中断也能被触发而不是等到退出临界段的时候才被触发，当然，这些中断服务函数中也不能调用 FreeRTOS 提供的 API 函数接口，而中断优先级在 5 到 15 的这些中断是可以被屏蔽的，也能安全调用 FreeRTOS 提供的 API 函数接口。

ARM Cortex-M NVIC 支持中断嵌套功能：当一个中断触发并且系统进行响应时，处理器硬件会将当前运行的部分上下文寄存器自动压入中断栈中，这部分的寄存器包括 PSR，R0，R1，R2，R3 以及 R12 寄存器。当系统正在服务一个中断时，如果有一个更高优先级的中断触发，那么处理器同样的会打断当前运行的中断服务例程，然后把老的中断服务例程上下文的 PSR，R0，R1，R2，R3 和 R12 寄存器自动保存到中断栈中。这些部分上下文寄存器保存到中断栈的行为完全是硬件行为，这一点是与其他 ARM 处理器最大的区别（以往都需要依赖于软件保存上下文）。

另外，在 ARM Cortex-M 系列处理器上，所有中断都采用中断向量表的方式进行处理，即当一个中断触发时，处理器将直接判定是哪个中断源，然后直接跳转到相应的固定位置进行处理。而在 ARM7、ARM9 中，一般是先跳转进入 IRQ 入口，然后再由软件进行判断是哪个中断源触发，获得了相对应的中断服务例程入口地址后，再进行后续的中断处理。ARM7、ARM9 的好处在于，所有中断它们都有统一的入口地址，便于 OS 的统一管理。而 ARM Cortex-M 系列处理器则恰恰相反，每个中断服务例程必须排列在一起放在统一的地 址上（这个地址必须要设置到 NVIC 的中断向量偏移寄存器中）。中断向量表一般由一个数组定义（或在起始代码中给出），在 STM32 上，默认采用起始代码给出：具体见代码。

```
1.  __Vectors          DCD      __initial_sp          ; Top of Stack
2.                                DCD      Reset_Handler      ; Reset Handler
3.                                DCD      NMI_Handler        ; NMI Handler
4.                                DCD      HardFault_Handler  ; Hard Fault Handler
5.                                DCD      MemManage_Handler  ; MPU Fault Handler
6.                                DCD      BusFault_Handler   ; Bus Fault Handler
7.                                DCD      UsageFault_Handler ; Usage Fault Handler
8.                                DCD      0                  ; Reserved
9.                                DCD      0                  ; Reserved
10.                               DCD      0                  ; Reserved
11.                               DCD      0                  ; Reserved
12.                               DCD      SVC_Handler        ; SVC Call Handler
13.                               DCD      DebugMon_Handler   ; Debug Monitor Handler
14.                               DCD      0                  ; Reserved
15.                               DCD      PendSV_Handler     ; PendSV Handler
16.                               DCD      SysTick_Handler    ; SysTick Handler
17.
18.                               ; External Interrupts
19.                               DCD      WWDG_IRQHandler     ; Window Watchdog
20.                               DCD      PVD_IRQHandler     ; PVD through EXTI Line detect
21.                               DCD      TAMPER_IRQHandler  ; Tamper
22.                               DCD      RTC_IRQHandler     ; RTC
23.                               DCD      FLASH_IRQHandler   ; Flash
24.                               DCD      RCC_IRQHandler     ; RCC
25.                               DCD      EXTI0_IRQHandler   ; EXTI Line 0
26.                               DCD      EXTI1_IRQHandler   ; EXTI Line 1
27.                               DCD      EXTI2_IRQHandler   ; EXTI Line 2
28.                               DCD      EXTI3_IRQHandler   ; EXTI Line 3
29.                               DCD      EXTI4_IRQHandler   ; EXTI Line 4
30.                               DCD      DMA1_Channel1_IRQHandler ; DMA1 Channel 1
31.                               DCD      DMA1_Channel2_IRQHandler ; DMA1 Channel 2
32.                               DCD      DMA1_Channel3_IRQHandler ; DMA1 Channel 3
33.                               DCD      DMA1_Channel4_IRQHandler ; DMA1 Channel 4
34.                               DCD      DMA1_Channel5_IRQHandler ; DMA1 Channel 5
35.                               DCD      DMA1_Channel6_IRQHandler ; DMA1 Channel 6
36.                               DCD      DMA1_Channel7_IRQHandler ; DMA1 Channel 7
37.                               DCD      ADC1_2_IRQHandler  ; ADC1 & ADC2
```

```

38.          DCD      USB_HP_CAN1_TX_IRQHandler ; USB High Priority or CAN1 TX
39.          DCD      USB_LP_CAN1_RX0_IRQHandler ; USB Low Priority or CAN1 RX0
40.          DCD      CAN1_RX1_IRQHandler ; CAN1 RX1
41.          DCD      CAN1_SCE_IRQHandler ; CAN1 SCE
42.          DCD      EXTI9_5_IRQHandler ; EXTI Line 9..5
43.          DCD      TIM1_BRK_IRQHandler ; TIM1 Break
44.          DCD      TIM1_UP_IRQHandler ; TIM1 Update
45.          DCD      TIM1_TRG_COM_IRQHandler ; TIM1 Trigger and Commutation
46.          DCD      TIM1_CC_IRQHandler ; TIM1 Capture Compare
47.          DCD      TIM2_IRQHandler ; TIM2
48.          DCD      TIM3_IRQHandler ; TIM3
49.          DCD      TIM4_IRQHandler ; TIM4
50.          DCD      I2C1_EV_IRQHandler ; I2C1 Event
51.          DCD      I2C1_ER_IRQHandler ; I2C1 Error
52.          DCD      I2C2_EV_IRQHandler ; I2C2 Event
53.          DCD      I2C2_ER_IRQHandler ; I2C2 Error
54.          DCD      SPI1_IRQHandler ; SPI1
55.          DCD      SPI2_IRQHandler ; SPI2
56.          DCD      USART1_IRQHandler ; USART1
57.          DCD      USART2_IRQHandler ; USART2
58.          DCD      USART3_IRQHandler ; USART3
59.          DCD      EXTI15_10_IRQHandler ; EXTI Line 15..10
60.          DCD      RTCAlarm_IRQHandler ; RTC Alarm through EXTI Line
61.          DCD      USBWakeUp_IRQHandler ; USB Wakeup from suspend
62.          DCD      TIM8_BRK_IRQHandler ; TIM8 Break
63.          DCD      TIM8_UP_IRQHandler ; TIM8 Update
64.          DCD      TIM8_TRG_COM_IRQHandler ; TIM8 Trigger and Commutation
65.          DCD      TIM8_CC_IRQHandler ; TIM8 Capture Compare
66.          DCD      ADC3_IRQHandler ; ADC3
67.          DCD      FSMC_IRQHandler ; FSMC
68.          DCD      SDIO_IRQHandler ; SDIO
69.          DCD      TIM5_IRQHandler ; TIM5
70.          DCD      SPI3_IRQHandler ; SPI3
71.          DCD      UART4_IRQHandler ; UART4
72.          DCD      UART5_IRQHandler ; UART5
73.          DCD      TIM6_IRQHandler ; TIM6
74.          DCD      TIM7_IRQHandler ; TIM7
75.          DCD      DMA2_Channel1_IRQHandler ; DMA2 Channel1
76.          DCD      DMA2_Channel2_IRQHandler ; DMA2 Channel2
77.          DCD      DMA2_Channel3_IRQHandler ; DMA2 Channel3
78.          DCD      DMA2_Channel4_5_IRQHandler ; DMA2 Channel4 & Channel5
79.  __Vectors_End

```



FreeRTOS 在 Cortex-M 系列处理器上也遵循与裸机中断一致的方法，当用户需要使用自定义的中断服务例程时，只需要定义相同名称的函数覆盖弱化符号即可。所以，FreeRTOS 在 Cortex-M 系列处理器的中断控制其实与裸机没什么差别。

八、中断管理实验

中断管理实验是在 FreeRTOS 中创建了两个任务分别获取信号量与消息队列，并且定义了两个按键 KEY1 与 KEY2 的触发方式为中断触发，其触发的中断服务函数则跟裸机一样，在中断触发的时候通过消息队列将消息传递给任务，任务接收到消息就将信息通过串口调试助手显示出来。而且中断管理实验也实现了一个串口的 DMA 传输+空闲中断功能，当串口接收完不定长的数据之后产生一个空闲中断，在中断中将信号量传递给任务，任务在收到信号量的时候将串口的数据读取出来并且在串口调试助手回显。

```

1.  /* FreeRTOS头文件 */
2.  #include "FreeRTOS.h"
3.  #include "task.h"
4.  #include "queue.h"
5.  #include "semphr.h"
6.
7.  /* 开发板硬件bsp头文件 */
8.  #include "bsp_led.h"
9.  #include "bsp_usart.h"
10. #include "bsp_key.h"
11. #include "bsp_exti.h"
12.
13. /* 标准库头文件 */
14. #include <string.h>
15.
16. /***** 任务句柄 *****/
17. /*
18.  * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
19.  * 以后我们想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
20.  * 这个句柄可以为NULL。
21.  */
22. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
23. static TaskHandle_t LED_Task_Handle = NULL; /* LED任务句柄 */

```

```

24. static TaskHandle_t KEY_Task_Handle = NULL; /* KEY任务句柄 */
25.
26. /***** 内核对象句柄 *****/
27. /*
28. * 信号量，消息队列，事件标志组，软件定时器这些都属于内核的对象，要想使用这些内核
29. * 对象，必须先创建，创建成功之后会返回一个相应的句柄。实际上就是一个指针，后续我
30. * 们就可以通过这个句柄操作这些内核对象。
31. *
32. * 内核对象说白了就是一种全局的数据结构，通过这些数据结构我们可以实现任务间的通信，
33. * 任务间的事件同步等各种功能。至于这些功能的实现我们是通过调用这些内核对象的函数
34. * 来完成的
35. *
36. */
37. QueueHandle_t Test_Queue = NULL;
38. SemaphoreHandle_t BinarySem_Handle = NULL;
39.
40. /***** 全局变量声明 *****/
41. /*
42. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
43. */
44.
45. extern char Usart_Rx_Buf[USART_RBUF_SIZE];
46.
47.
48. /***** 宏定义 *****/
49. /*
50. * 当我们在写应用程序的时候，可能需要用到一些宏定义。
51. */
52. #define QUEUE_LEN 4 /* 队列的长度，最大可包含多少个消息 */
53. #define QUEUE_SIZE 4 /* 队列中每个消息大小（字节） */
54.
55.
56. /*
57. *****/
58. * 函数声明
59. *****/
60. /*
61. static void AppTaskCreate(void); /* 用于创建任务 */
62.
63. static void LED_Task(void* pvParameters); /* LED_Task任务实现 */
64. static void KEY_Task(void* pvParameters); /* KEY_Task任务实现 */
65.
66. static void BSP_Init(void); /* 用于初始化板载相关资源 */
67.
68. /*****
69. * @brief 主函数
70. * @param 无
71. * @retval 无
72. * @note 第一步：开发板硬件初始化
73. 第二步：创建APP应用任务
74. 第三步：启动FreeRTOS，开始多任务调度
75. *****/
76. int main(void)
77. {
78. BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
79.
80. /* 开发板硬件初始化 */
81. BSP_Init();
82.
83. printf("这是一个FreeRTOS中断管理实验！\n");
84. printf("按下KEY1 | KEY2触发中断！\n");
85. printf("串口发送数据触发中断，任务处理数据!\n");
86.
87. /* 创建AppTaskCreate任务 */
88. xReturn = xTaskCreate((TaskFunction_t)AppTaskCreate, /* 任务入口函数 */
89. (const char*) "AppTaskCreate", /* 任务名字 */
90. (uint16_t) 512, /* 任务栈大小 */
91. (void*) NULL, /* 任务入口函数参数 */
92. (UBaseType_t) 1, /* 任务的优先级 */
93. (TaskHandle_t*) &AppTaskCreate_Handle); /* 任务控制块指针 */
94. /* 启动任务调度 */
95. if(pdPASS == xReturn)
96. vTaskStartScheduler(); /* 启动任务，开启调度 */
97. else
98. return -1;
99. //
100. while(1); /* 正常不会执行到这里 */
101. }
102.
103.
104. /*****
105. * @ 函数名 : AppTaskCreate
106. * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
107. * @ 参数 : 无
108. * @ 返回值 : 无
109. *****/

```

```

110. static void AppTaskCreate(void)
111. {
112.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
113.
114.     taskENTER_CRITICAL();          //进入临界区
115.
116.     /* 创建Test_Queue */
117.     Test_Queue = xQueueCreate((UBaseType_t ) QUEUE_LEN, /* 消息队列的长度 */
118.                               (UBaseType_t ) QUEUE_SIZE); /* 消息的大小 */
119.
120.     if(NULL != Test_Queue)
121.         printf("Test_Queue消息队列创建成功!\n");
122.
123.     /* 创建 BinarySem */
124.     BinarySem_Handle = xSemaphoreCreateBinary();
125.
126.     if(NULL != BinarySem_Handle)
127.         printf("BinarySem_Handle二值信号量创建成功!\n");
128.
129.     /* 创建LED_Task任务 */
130.     xReturn = xTaskCreate((TaskFunction_t )LED_Task, /* 任务入口函数 */
131.                          (const char* )"LED_Task", /* 任务名字 */
132.                          (uint16_t )512, /* 任务栈大小 */
133.                          (void* )NULL, /* 任务入口函数参数 */
134.                          (UBaseType_t )2, /* 任务的优先级 */
135.                          (TaskHandle_t* )&LED_Task_Handle); /* 任务控制块指针 */
136.     if(pdPASS == xReturn)
137.         printf("创建LED_Task任务成功!\n");
138.     /* 创建KEY_Task任务 */
139.     xReturn = xTaskCreate((TaskFunction_t )KEY_Task, /* 任务入口函数 */
140.                          (const char* )"KEY_Task", /* 任务名字 */
141.                          (uint16_t )512, /* 任务栈大小 */
142.                          (void* )NULL, /* 任务入口函数参数 */
143.                          (UBaseType_t )3, /* 任务的优先级 */
144.                          (TaskHandle_t* )&KEY_Task_Handle); /* 任务控制块指针 */
145.     if(pdPASS == xReturn)
146.         printf("创建KEY_Task任务成功!\n");
147.
148.     vTaskDelete(AppTaskCreate_Handle); //删除AppTaskCreate任务
149.
150.     taskEXIT_CRITICAL();          //退出临界区
151. }
152.
153.
154.
155. /*****
156.  * @ 函数名 : LED_Task
157.  * @ 功能说明: LED_Task任务主体
158.  * @ 参数 :
159.  * @ 返回值 : 无
160.  *****/
161. static void LED_Task(void* parameter)
162. {
163.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
164.     uint32_t r_queue; /* 定义一个接收消息的变量 */
165.     while (1)
166.     {
167.         /* 队列读取（接收），等待时间为一直等待 */
168.         xReturn = xQueueReceive( Test_Queue, /* 消息队列的句柄 */
169.                                 &r_queue, /* 发送的消息内容 */
170.                                 portMAX_DELAY); /* 等待时间 一直等 */
171.
172.         if(pdPASS == xReturn)
173.         {
174.             printf("触发中断的是 KEY%d !\n", r_queue);
175.         }
176.         else
177.         {
178.             printf("数据接收出错\n");
179.         }
180.
181.         LED1_TOGGLE;
182.     }
183. }
184.
185. /*****
186.  * @ 函数名 : LED_Task
187.  * @ 功能说明: LED_Task任务主体
188.  * @ 参数 :
189.  * @ 返回值 : 无
190.  *****/
191. static void KEY_Task(void* parameter)
192. {
193.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
194.     while (1)
195.     {

```



```

196. //获取二值信号量 xSemaphore,没获取到则一直等待
197.         xReturn = xSemaphoreTake(BinarySem_Handle,/* 二值信号量句柄 */
198.                                   portMAX_DELAY); /* 等待时间 */
199.     if(pdPASS == xReturn)
200.     {
201.         printf("收到数据:%s\n",Usart_Rx_Buf);
202.         memset(Usart_Rx_Buf,0,USART_RBUFF_SIZE);/* 清零 */
203.         LED2_TOGGLE;
204.     }
205. }
206. }
207.
208. /*****
209.  * @ 函数名 : BSP_Init
210.  * @ 功能说明: 板级外设初始化,所有板子上的初始化均可放在这个函数里面
211.  * @ 参数 :
212.  * @ 返回值 : 无
213.  *****/
214. static void BSP_Init(void)
215. {
216.     /*
217.      * STM32中断优先级分组为4,即4bit都用来表示抢占优先级,范围为:0~15
218.      * 优先级分组只需要分组一次即可,以后如果有其他的任务需要用到中断,
219.      * 都统一用这个优先级分组,千万不要再分组,切忌。
220.      */
221.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
222.
223.     /* LED 初始化 */
224.     LED_GPIO_Config();
225.
226.     /* DMA初始化 */
227.     USARTx_DMA_Config();
228.
229.     /* 串口初始化 */
230.     USART_Config();
231.
232.     /* 按键初始化 */
233.     Key_GPIO_Config();
234.
235.     /* 按键初始化 */
236.     EXTI_Key_Config();
237.
238. }
239.
240. /*****END OF FILE*****/

```



中断服务程序

```

1. /* Includes -----*/
2. #include "stm32f10x_it.h"
3.
4. /* FreeRTOS头文件 */
5. #include "FreeRTOS.h"
6. #include "task.h"
7. #include "queue.h"
8. #include "semphr.h"
9. /* 开发板硬件bsp头文件 */
10. #include "bsp_led.h"
11. #include "bsp_usart.h"
12. #include "bsp_key.h"
13. #include "bsp_exti.h"
14.
15. /**
16.  * @brief This function handles SysTick Handler.
17.  * @param None
18.  * @retval None
19.  */
20. extern void xPortSysTickHandler(void);
21. //systick中断服务函数
22. void SysTick_Handler(void)
23. {
24.     #if (INCLUDE_xTaskGetSchedulerState == 1 )
25.         if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED)
26.         {
27.             #endif /* INCLUDE_xTaskGetSchedulerState */
28.
29.             xPortSysTickHandler();
30.
31.             #if (INCLUDE_xTaskGetSchedulerState == 1 )
32.             }
33.             #endif /* INCLUDE_xTaskGetSchedulerState */
34.         }
35.

```

```

36.
37.
38. /* 声明引用外部队列 & 二值信号量 */
39. extern QueueHandle_t Test_Queue;
40. extern SemaphoreHandle_t BinarySem_Handle;
41.
42. static uint32_t send_data1 = 1;
43. static uint32_t send_data2 = 2;
44.
45. /*****
46.  * @ 函数名 : KEY1_IRQHandler
47.  * @ 功能说明: 中断服务函数
48.  * @ 参数 : 无
49.  * @ 返回值 : 无
50.  *****/
51. void KEY1_IRQHandler(void)
52. {
53.     BaseType_t pxHigherPriorityTaskWoken;
54.     //确保是否产生了EXTI Line中断
55.     uint32_t ulReturn;
56.     /* 进入临界段, 临界段可以嵌套 */
57.     ulReturn = taskENTER_CRITICAL_FROM_ISR();
58.
59.     if(EXTI_GetITStatus(KEY1_INT_EXTI_LINE) != RESET)
60.     {
61.         /* 将数据写入(发送)到队列中, 等待时间为 0 */
62.         xQueueSendFromISR(Test_Queue, /* 消息队列的句柄 */
63.                             &send_data1, /* 发送的消息内容 */
64.                             &pxHigherPriorityTaskWoken);
65.
66.         //如果需要的话进行一次任务切换
67.         portYIELD_FROM_ISR(pxHigherPriorityTaskWoken);
68.
69.         //清除中断标志位
70.         EXTI_ClearITPendingBit(KEY1_INT_EXTI_LINE);
71.     }
72.
73.     /* 退出临界段 */
74.     taskEXIT_CRITICAL_FROM_ISR( ulReturn );
75. }
76.
77. /*****
78.  * @ 函数名 : KEY1_IRQHandler
79.  * @ 功能说明: 中断服务函数
80.  * @ 参数 : 无
81.  * @ 返回值 : 无
82.  *****/
83. void KEY2_IRQHandler(void)
84. {
85.     BaseType_t pxHigherPriorityTaskWoken;
86.     uint32_t ulReturn;
87.     /* 进入临界段, 临界段可以嵌套 */
88.     ulReturn = taskENTER_CRITICAL_FROM_ISR();
89.
90.     //确保是否产生了EXTI Line中断
91.     if(EXTI_GetITStatus(KEY2_INT_EXTI_LINE) != RESET)
92.     {
93.         /* 将数据写入(发送)到队列中, 等待时间为 0 */
94.         xQueueSendFromISR(Test_Queue, /* 消息队列的句柄 */
95.                             &send_data2, /* 发送的消息内容 */
96.                             &pxHigherPriorityTaskWoken);
97.
98.         //如果需要的话进行一次任务切换
99.         portYIELD_FROM_ISR(pxHigherPriorityTaskWoken);
100.
101.         //清除中断标志位
102.         EXTI_ClearITPendingBit(KEY2_INT_EXTI_LINE);
103.     }
104.
105.     /* 退出临界段 */
106.     taskEXIT_CRITICAL_FROM_ISR( ulReturn );
107. }
108.
109. /*****
110.  * @ 函数名 : DEBUG_USART_IRQHandler
111.  * @ 功能说明: 串口中断服务函数
112.  * @ 参数 : 无
113.  * @ 返回值 : 无
114.  *****/
115. void DEBUG_USART_IRQHandler(void)
116. {
117.     uint32_t ulReturn;
118.     /* 进入临界段, 临界段可以嵌套 */
119.     ulReturn = taskENTER_CRITICAL_FROM_ISR();
120.
121.     if(USART_GetITStatus(DEBUG_USARTx, USART_IT_IDLE) != RESET)

```

```

122.     {
123.         Uart_DMA_Rx_Data();          /* 释放一个信号量，表示数据已接收 */
124.         USART_ReceiveData(DEBUG_USARTx); /* 清除标志位 */
125.     }
126.
127. /* 退出临界段 */
128. taskEXIT_CRITICAL_FROM_ISR( ulReturn );
129. }
130.
131.

```



九、中断管理实验现象

程序编译好，用 USB 线连接电脑和开发板的 USB 接口（对应丝印为 USB 转串口），用 仿真器把配套程序下载到 STM32 开发板，在电脑上打开串口调试助手，然后复位开发板就可以在调试助手中看到串口的打印信息，按下开发板的 KEY1 按键触发中断发送消息 1，按下 KEY2 按键发送消息 2；我们按下 KEY1 与 KEY2 试试，在串口调试助手中可以看到运行结果，然后通过串口调试助手发送一段不定长信息，触发中断会在中断服务函数发送信号量通知任务，任务接收到信号量的时候将串口信息打印出来。

串口配置

端口

COM13 US1

波特率

115200

数据位

8

停止位

1

校验位

无

操作

关闭串口

接收区设置

ASCII

HEX

停止显示

日志模式

清空接收区

保存到文件

这是一个FreeRTOS中断管理实验！

按下KEY1 | KEY2触发中断！

串口发送数据触发中断,任务处理数据！

Test_Queue消息队列创建成功！

BinarySem_Handle二值信号量创建成功！

创建LED_Task任务成功！

创建KEY_Task任务成功！

触发中断的是 KEY1！

触发中断的是 KEY1！

触发中断的是 KEY2！

触发中断的是 KEY2！

收到数据:串口DMA接收数据，接收完数据就触发串口空闲中断

触发中断的是 KEY2！

触发中断的是 KEY1！

收到数据:串口DMA接收数据，接收完数据就触发串口空闲中断

触发中断的是 KEY1！

串口DMA接收数据，接收完数据就触发串口空闲中断

CSDN @不秃也很强